

Specific Objectives

1. Demonstrate your abilities at following an agile methodology to develop a software system as a team while learning software tools in an autonomous way.

Student Outcome

3. Communicate effectively in a variety of professional contexts.
6. Apply computer science theory and software development fundamentals to produce computing-based solutions.

Description

Develop the software system selected and designed in our previous module of Software Engineering I, using a development methodology and software tools to achieve a final product appropriately tested and versioned with a clean code. The system must be developed in teams composed of 3 to 5 members.

The deployment to production will be monitored and this deployment is only accepted after the first sprint. At the end of each sprint, you are expected to provide:

- An acceptance form signed by the customer.
- Evidence of sprint review
- The software system deployed into a production environment.

Reporting Communications

The name of your repository must be the same than your team (i.e., T1, T2, ..., Tn). This repository must contain a folder named *Communications*. This folder should include all the pieces of evidence of communication between you and your client. Some examples of pieces of evidence include: (i) instant messaging (e.g., WhatsApp, Signal); (ii) emails; (iii) voice messages or recordings; (iv) videos; and (vii) commitment/acceptance letter.

Create a table of content (readme) to index all the pieces of evidence within the communications folder. Each entrance in the index must specify (i) the name of the file, (ii) a brief description of its content; (iii) the file type; (iv) the participants; (v) date and time, among others. The document must be constantly updated until the deployment of the final version of the software system.

Section A: Project Report and Implementation

Each team must write a project report, in English, that adheres to the template and guidelines specified in the reference “Writing guidelines and templates” [3] from ETS but using ESPOL logo [6]. The report must be a **self-contained document** with pieces of evidence about the project context, development process, testing levels, deployment, etc.

Section B: Project Presentation

The team must perform a live demonstration (in English) of your software system in the classroom for the lecturer, classmates, university staff, and even customers. You must record the demonstration and upload it -in less than 24 hours- to a repository specified by the lecturer. The demonstration is expected to last 13 minutes and include equal participation of all team members. The team must show the software components running and demonstrate their compliance with the functional and non-functional requirements. The team must also present the results of the evaluation of the system, covering aspects of your system design and other points covered in the project report, including a review of your test plans. The team is also encouraged to upload the presentation file used during your demonstration.

Some aspects that your presentation must include are:

1. a brief introduction of the system (who the client is, system scope, what the system does).
2. The number of user stories the system has.
3. The number of sprints needed to develop the system.
4. The architecture (deployment and component diagrams).
5. How users interact with the system by performing test cases (acceptance testing based on the user stories and acceptance criteria). Be sure that one user per role appears in the demonstration.
6. Other information you consider important.

The presentation must be prepared using ESPOL official templates [6], attached to this specification.

Deliverables

1. A **self-contained report** of the project (**in pdf**) written in English that includes (the uploaded file must be named as T<team number><ProjectName>.pdf):
 - a. a compilation of relevant documents, charts, reports, and any other information that supports your work in reference to the elements assessed in the marking scheme.
 - b. The repository's URL with the source code, resources, communication, and SCRUM evidence.
 - c. Access to any online tool to support any of the elements assessed in the marking scheme.
 - d. An acceptance form signed by the customer.
 - e. The recorded video of your project presentation.
 - f. A section titled "Individual Contributions", in which the contributions of each team member are specified [7, 8].
2. The software system deployed into a production environment.
3. Each student must submit an evaluation of other team members using the co-evaluation scheme in the individual assignment "Project Co-evaluation", available in Aula Virtual.

Marking Scheme

Presentation	Weight
Presentation of project information details (client, objectives, scope, scenarios, etc.)	03
Justification of the relevant architectural decisions made (including selected tools/frameworks)	03
Demonstration of the use of the application features	10
Presentation of testing management and documentation	04
Report and Implementation	
Appendixes with user manual [5]	10
Appendixes with installation guide [4]	10
Pieces of evidence about project details, communication, and architectural decisions	10
Risk-based testing and test documentation	10
Evidence of adherence to SCRUM [1]: backlogs, sprint reviews, planning, charts, user stories, etc.	05
Enforcing coding standards (e.g., Checkstyle) and Preemptive error detection (e.g., PMD)	10
Use of a continuous integration tool (e.g., Jenkins) with quality data	10
Use of a tool to perform acceptance testing (e.g., Cucumber)	10
Compliance with SOLID principles, design patterns, and refactoring	05
Total	100
Extras	
Application profiling (e.g., JProfiler, YourKit)	01
Use of a GUI test automation tool (e.g., Selenium [2])	01
Load testing (e.g., Apache JMeter)	01
Penalties	
Lack of system deployment into a production environment	-100
Lack of evidence of communications over time as requested	-30
Absence of client acceptance form	-100
Non-compliance with a delivery standard or specification	-100%
Each grammatical or orthographic error (maximum 20 mistakes)	-01
Non-compliance with the corresponding co-evaluation assignment (individual involvement)	-50
Repository without access permission	-50

Late Submission Policy

Delay (§)	Penalty (Ω)
12 hours or less	loss of 10%
12 to 24 hours	loss of 20%
24 to 48 hours	loss of 30%
Over 48 hours:	loss of 100%

(§) every clock hour counts including weekends or holidays.

(Ω) automatic and non-negotiable penalty.

References

1. Kenneth S. Rubin, "Essential SCRUM: A Practical Guide to the Most Popular Agile Process", 2012, Addison-Wesley
2. Selenium, <https://www.selenium.dev>
3. ETS, "Writing guidelines and templates", <https://www.etsmtl.ca/en/studies/Guidelines-and-templates>
4. Klariti, "How to Write a Software Installation Guide", <https://klariti.com/2018/02/20/how-to-write-an-installation-guide-with-sample-guidelines/>
5. TechSmith, "How to Build the Best User Documentation", <https://www.techsmith.com/blog/user-documentation/>
6. FIEC, "Identidad FIEC", <https://www.fiec.espol.edu.ec/archive/es/identidad-fiec>
7. ACM, "ACM Policy on Authorship", <https://www.acm.org/publications/policies/new-acm-policy-on-authorship>
8. ACM, "Frequently Asked Questions", <https://www.acm.org/publications/policies/frequently-asked-questions>

Assessment Criteria for Student Outcome 3

Criteria	Unsatisfactory	Developing	Satisfactory	Exemplary
Oral and Written Presentation Structure	Lacks logical organization. It shows some coherence, but ideas lack coherence. Vocabulary is limited, casual, and colloquial.	Some information presented in logical sequence. Some points remain displaced and stray from the topic. Vocabulary is sometimes vague.	Is coherent and logically organized. Transitions used between ideas and paragraphs to create coherence. Vocabulary is clear.	Shows high degree of attention to logic and reasoning of points. Clearly leads the reader/listener to the conclusion and stirs thought regarding the topic. Vocabulary is highly appropriate for the context. It is very clear.
Written presentation /Task achievement	Is not logical, Ideas are present with details that may be irrelevant, inappropriate, or inaccurate. Continuous errors in spelling, grammar, syntax and/or punctuation making it difficult to read.	Is logical and solid. Ideas are present but could be more fully extended. Shows a pattern of errors in spelling, grammar, syntax and/or punctuation.	Is well-presented. Ideas are detailed, developed and supported. Has a few errors in spelling, grammar, syntax and punctuation.	Is exceptionally well presented and argued. Ideas are detailed, well developed and supported with specific evidence and facts. Is free of distracting spelling, grammar, syntax and punctuation errors.
Oral Presentation Fluency and Pronunciation	Speech is slow, hesitant & strained except for short, memorized phrases; inaudible. More than three mispronounced words	Speech is relatively smooth with some hesitation and unevenness. Volume wavers. No more than two mispronounced words.	Fluid speech; few hesitations; a slight search for words. Inaudible word or two. No more than one mispronounced word.	Fluid speech; few to no hesitations; no attempts to search for words. Volume is excellent. No mispronounced words.

Assessment Criteria for Student Outcome 6

Criteria	Unsatisfactory	Developing	Satisfactory	Exemplary
<i>Selects an appropriate solution based on computer science theory for a given problem.</i>	Does not propose any appropriate solution based on computer science theory.	Proposes an appropriate solution, but incorrectly applying computer science theory.	Proposes an appropriate solution based on computer science theory in an effective manner.	Proposes an appropriate solution based on computer science theory in an effective and efficient or innovative manner.
<i>Implements the appropriate solution using software development fundamentals.</i>	The implemented solution does not comply with any of studied software development fundamentals.	More than one software development fundamental is applied incorrectly in the implementation of the selected solution.	Applies software development fundamentals correctly in the implementation of selected solution.	Applies software development fundamentals correctly in the implementation of the selected solution and proposes future improvements.